WFLOW-06: Custom Notification Templates

> **Series:** WFLOW — Workflows and Alert Notifications | **Notebook:** 6 of 10 | **Created:** January 2026 | **Last Updated:** 11/11/2026

Rich Message Formatting

Create professional, informative notifications with dynamic content, formatting, and data enrichment. This notebook covers Jinja templating, Slack Block Kit, Teams Adaptive Cards, and data enrichment patterns.

Table of Contents

1. Template Design Principles
2. Jinja2 Expression Deep Dive
3. Slack Block Kit Templates
4. Teams Adaptive Card Templates
5. Data Enrichment
6. Template Library
7. Testing Templates

Prerequisites

Requirement	Details
Dynatrace Environment	SaaS with Platform subscription
Permissions	`automation:workflows:write`
Prior Knowledge	**WFLOW-01** through **WFLOW-05**
Slack	A Slack connection (Dynatrace Slack app) for the Slack templates
Microsoft Teams	A Power Automate *Send webhook alerts to a channel* webhook URL as the Teams connection, and its host (for example `*.api.powerplatform.com`) under **Settings > General > External requests**
Problem trigger	`storage:events:read` for the workflow actor

1. Template Design Principles

Effective Alert Messages

Principle	Good	Bad
Scannable	Key info in first line	Important details buried
Actionable	Include link to problem	"Check Dynatrace"
Contextual	Show affected service	Generic "problem detected"
Severity-coded	Visual severity indicator	Plain text only
Concise	Essential details only	Long descriptions

Message Structure

```
[1] SEVERITY INDICATOR + TITLE
[2] Key metrics/impact
[3] Affected entities
[4] Action button/link
```

Severity Visual Guide

`event.severity`	Slack	Teams	Email
1 (Critical)	:red_circle:	Red header	Red banner
2 (Major)	:large_orange_circle:	Orange header	Orange banner
3 (Minor)	:large_yellow_circle:	Yellow header	Yellow banner
4 (Warning)	:large_blue_circle:	Blue header	Blue banner

The problem record carries severity as `event.severity` on a 1–5 scale (`stable` in the semantic dictionary, read 11/11/2026), not as `CRITICAL` / `HIGH` strings. Dynatrace names the levels Critical, Major, Minor, Warning and Informational, and problems carry only the first three: *"Warning and Informational events appear in the event list and can be grouped into a problem by problem correlation, but they don't raise problems on their own."* The Warning row matters only for templates that also serve Davis events. The templates below convert it with `| int(5)`, so a missing severity renders as the lowest tier; from SaaS 1.348 severity is no longer defaulted (WFLOW-04 §3). WFLOW-02 §7 maps the other old field names.

> **Sources:** Standardized event severity (DT docs).

2. Jinja2 Expression Deep Dive

Variable Access

```
{{ event()["event.name"] }}         {# Direct field access #}
{{ event().get("field") or "default" }} {# With default value; also covers a
present-but-null field #}
{{ result("task_name").output }}    {# Previous task result #}
{{ environment().url }}              {# Environment URL #}
{{ problem_link() }}                 {# Link to the problem (Problem trigger
only) #}
{{ now() }}                          {# Current timestamp #}
```

Filters

```
{{ event()["event.name"] | upper }}           {# UPPERCASE #}
{{ event()["event.name"] | lower }}           {# lowercase #}
{{ event()["event.name"] | truncate(50) }}    {# Limit length #}
{{ list_field | join(", ") }}            {# Join array #}
{{ number | round(2) }}                   {# Round decimals #}
{{ now("Europe/Vienna").strftime("%Y-%m-%d %H:%M") }}  {# Format a date:
strftime on a datetime #}
```

There is no `format_datetime` filter. `now()` returns a datetime object, formatted with `strftime`: *"Use strftime and format codes as described in datetime Python documentation"*. For a timestamp string such as `event.start`, the `to_datetime` filter returns a datetime; test it in the expression preview, because problem timestamps carry nanoseconds.

> **Deprecated problem fields.** The templates in this notebook read `affected_entity_ids` and `root_cause_entity_id`, which still work. The upgrade guide marks them deprecated: *"root_cause_entity_id and root_cause_entity_name are deprecated in favor of root_cause.smartscape_entity , and affected_entity_ids in favor of smartscape.affected_entities ."* The classic fields were the better-populated ones on the validation tenant (10/06/2026), so read the Smartscape field first and fall back, as in `{{ (event().get("smartscape.affected_entities") or []) | map(attribute="name") | join(", ") or event()["affected_entity_ids"] | join(", ") }}`. **WFLOW-05 §3** has the counts and a root-cause example.

> **Sources:** Jinja expressions for Workflows (DT docs); Upgrade guide — alert notification (DT docs).

Conditionals

```
{% set sev = event().get("event.severity") | int(5) %}
{% if sev &lt;= 1 %}
  :rotating_light: CRITICAL ALERT
{% elif sev == 2 %}
  :warning: MAJOR ALERT
{% else %}
  :information_source: {{ event()["event.category"] }} ALERT
{% endif %}
```

Loops

```
Affected Entities:
{% for entity in event()["affected_entity_ids"][:5] %}
  - {{ entity }}
{% endfor %}
{% if event()["affected_entity_ids"] | length &gt; 5 %}
  ... and {{ event()["affected_entity_ids"] | length - 5 }} more
{% endif %}
```

Inline Conditionals

```
{{ ":red_circle:" if (event().get("event.severity") | int(5)) &lt;= 1 else
":large_yellow_circle:" }}

{{ event().get("root_cause_entity_id") or "Pending analysis" }}
```

Dictionary Mapping

```
{# Severity (1-5) to emoji mapping #}
{{ {
    1: ":red_circle:",
    2: ":large_orange_circle:",
    3: ":large_yellow_circle:",
    4: ":large_blue_circle:"
   }.get(event().get("event.severity") | int(5), ":white_circle:") }}
```

3. Slack Block Kit Templates

Full-Featured Alert Template

The Slack **Send message** action (`dynatrace.slack:slack-send-message`) has no separate `blocks` input. With `messageFormat: slack_format`, *"Slack Markdown or Slack Block Kit inputs are processed"* from the `message` field. The layout below is written as YAML for readability: convert it to JSON (Slack's Block Kit Builder does this) and paste it into `message`. Send one test message, because expressions resolve before Slack parses the JSON, and a value containing a double quote breaks it.

> **Sources:** Slack Connector actions (DT docs) — *"If messageFormat is slack_format , Slack Markdown or Slack Block Kit inputs are processed."*

```
# Block Kit layout for the Send message task's `message` (as JSON)
blocks:
    # Header with severity
    - type: header
      text:
        type: plain_text
        text: "{{ {1: ':rotating_light:', 2: ':warning:', 3:
':large_yellow_circle:', 4:
':information_source:'}.get(event().get('event.severity') | int(5),
':grey_question:') }} {{ event()['event.category'] }} Problem"

    # Problem title
    - type: section
      text:
        type: mrkdwn
        text: "*{{ event()['event.name'] }}*"

    # Details in two columns
    - type: section
      fields:
        - type: mrkdwn
          text: "*Problem ID:*\n{{ event()['display_id'] }}"
        - type: mrkdwn
          text: "*Status:*\n{{ event()['event.status'] }}"
        - type: mrkdwn
          text: "*Started:*\n{{ event()['event.start'] }}"
        - type: mrkdwn
          text: "*Category:*\n{{ event()['event.category'] }}"

    # Root cause (if available)
    - type: section
      text:
        type: mrkdwn
        text: "*Root Cause:*\n{{ event().get('root_cause_entity_id', 'Analysis
in progress...') }}"

    # Affected entities
    - type: section
      text:
        type: mrkdwn
        text: "*Affected ({{ event()['affected_entity_ids'] | length }}):*\n{{
event()['affected_entity_ids'][:3] | join('\n') }}{{ '\n...' if event()
['affected_entity_ids'] | length &gt; 3 else '' }}"

    # Divider
    - type: divider
```

```
    # Action buttons
    - type: actions
      elements:
        - type: button
          text:
            type: plain_text
            text: "View Problem"
          url: "{{ problem_link() }}"
          style: primary
        - type: button
          text:
            type: plain_text
            text: "View Service"
          url: "{{ environment().url }}/ui/services"

    # Footer context
    - type: context
      elements:
        - type: mrkdwn
          text: "Detected by Dynatrace Intelligence | {{ now() }}"
```

4. Teams Adaptive Card Templates

Full-Featured Alert Card

The Microsoft Teams **Send message** action (`dynatrace.msteams:send-message`) takes the card as Adaptive Card JSON in its `message` field, with the connection in `connectionId`; alternatively, `selectTemplate` picks *"A pre-defined AdaptiveCard used as a message."* The connection is a Power Automate webhook (*"Office 365 webhooks will stop working."*), and its host must be allowed under External requests (see Prerequisites). The card below is written as YAML for readability: convert it to JSON and paste it into `message`. Use Dynatrace expressions, as here; the docs say *"We don't support Adaptive Cards Template Language templating"*.

> **Sources:** Microsoft Teams Connector (DT docs); threat-detection-notification-sender.yaml (Dynatrace GitHub) — *"action: dynatrace.msteams:send-message"*.

```
# Adaptive Card for the Send message task's `message` (as JSON)
card:
    type: AdaptiveCard
    $schema: "https://adaptivecards.microsoft.com/schemas/adaptive-card.json"
    version: "1.4"
    body:
      # Header with color
      - type: TextBlock
        text: "{{ event()['event.category'] }} Problem Detected"
        size: Large
        weight: Bolder
        color: "{{ {1: 'Attention', 2: 'Warning', 3: 'Accent', 4:
'Good'}.get(event().get('event.severity') | int(5), 'Default') }}"

      # Problem title
      - type: TextBlock
        text: "{{ event()['event.name'] }}"
        wrap: true
        weight: Bolder

      # Details table
      - type: FactSet
        facts:
          - title: "Problem ID"
            value: "{{ event()['display_id'] }}"
          - title: "Category"
            value: "{{ event()['event.category'] }}"
          - title: "Status"
            value: "{{ event()['event.status'] }}"
          - title: "Started"
            value: "{{ event()['event.start'] }}"
          - title: "Root Cause"
            value: "{{ event().get('root_cause_entity_id', 'Analyzing...') }}"

      # Affected entities
      - type: TextBlock
        text: "Affected Entities ({{ event()['affected_entity_ids'] | length
}})"
        weight: Bolder
        spacing: Medium
      - type: TextBlock
        text: "{{ event()['affected_entity_ids'][:5] | join(', ') }}{{ ', ...'
if event()['affected_entity_ids'] | length &gt; 5 else '' }}"
        wrap: true
        size: Small

    actions:
```

```
      - type: Action.OpenUrl
        title: "View in Dynatrace"
        url: "{{ problem_link() }}"
```

5. Data Enrichment

Enrich with DQL Query

Add recent error logs to notification:

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { queryExecutionClient } from '@dynatrace-sdk/client-query';

async function runQuery(query) {
  // queryExecute returns a requestToken instead of the result when the query
outlasts
  // requestTimeoutMilliseconds - poll until it reaches a final state (see
WFLOW-08 §2).
  const started = await queryExecutionClient.queryExecute({ body: { query,
requestTimeoutMilliseconds: 30000 } });
  let r = started;
  while (r &amp;amp;&amp;amp; (r.state === 'NOT_STARTED' || r.state === 'RUNNING')) {
    r = await queryExecutionClient.queryPoll({ requestToken:
started.requestToken, requestTimeoutMilliseconds: 30000 });
  }
  if (!r || r.state !== 'SUCCEEDED') throw new Error(`DQL query did not
succeed: ${r?.state ?? 'no response'}`);
  return r.result.records;
}

export default async function () {
  const event = (await execution()).params.event;   // trigger payload
  // Get recent error logs for the root-cause entity. root_cause_entity_id can
be any
  // entity type (on the validation tenant only ~6% were services), so match
the field for its type.
  const rootCause = event.root_cause_entity_id;

  if (!rootCause) {
    return { event, recent_errors: [] };
  }

  const recentErrors = await runQuery(`
    fetch logs, from: now() - 30m
    | filter dt.entity.service == "${rootCause}" or dt.entity.host ==
"${rootCause}"
          or dt.entity.process_group == "${rootCause}" or
dt.entity.process_group_instance == "${rootCause}"
          or dt.entity.cloud_application == "${rootCause}"
    | filter status == "ERROR"
    | fields timestamp, content
    | sort timestamp desc
    | limit 5
  `);

  return {
    event,
```

```
    recent_errors: recentErrors
  };
}
```

Use Enriched Data in Template

```
*Recent Error Logs:*
{% for log in result("enrich_data").recent_errors[:3] %}
• `{{ log.content | truncate(100) }}`
{% endfor %}
{% if result("enrich_data").recent_errors | length == 0 %}
_No error logs found for the root-cause entity in the last 30 minutes_
{% endif %}
```

Enrich with Entity Details

```
import { execution } from '@dynatrace-sdk/automation-utils';
import { monitoredEntitiesClient } from '@dynatrace-sdk/client-classic-
environment-v2';

export default async function () {
  const event = (await execution()).event();   // trigger payload, or null
  const entityId = event?.root_cause_entity_id;

  if (!entityId) {
    return { entity_name: 'Unknown', entity_type: 'Unknown' };
  }

  const entity = await monitoredEntitiesClient.getEntity({
    entityId: entityId
  });

  return {
    entity_name: entity.displayName,
    entity_type: entity.type,
    tags: entity.tags || []
  };
}
```

6. Template Library

Compact Alert (Slack)

```
message: |
  {{ {1: ":red_circle:", 2: ":large_orange_circle:", 3:
":large_yellow_circle:", 4:
":large_blue_circle:"}.get(event().get("event.severity") | int(5),
":white_circle:") }} *{{ event()["event.name"] }}*
  `{{ event()["display_id"] }}` | {{ event()["event.category"] }} | &lt;{{
problem_link() }}|View&gt;
```

Problem Resolved (Slack)

```
message: |
  :white_check_mark: *Problem Resolved*

  *{{ event()["event.name"] }}*

  • *Duration:* {{ ((event().get("resolved_problem_duration") | int(0)) /
60000000000) | round(1) }} min
  • *Problem ID:* {{ event()["display_id"] }}
  • *Resolved:* {{ event().get("event.end", now()) }}
```

Daily Summary (Scheduled)

```
message: |
  :bar_chart: *Daily Problem Summary*

  *Last 24 Hours:*
  • Problems opened: {{ result("query_stats").opened }}
  • Problems resolved: {{ result("query_stats").resolved }}
  • Currently open: {{ result("query_stats").open }}

  *By Severity:*
  :red_circle: Critical: {{ result("query_stats").critical }}
  :large_orange_circle: Major: {{ result("query_stats").major }}
  :large_yellow_circle: Minor: {{ result("query_stats").minor }}
```

Escalation Notice

```
message: |
  :rotating_light: *ESCALATION*

  Problem *{{ event()["display_id"] }}* is still active after 15 minutes.

  *{{ event()["event.name"] }}*

  This alert is being escalated to the on-call team.
  &lt;{{ problem_link() }}|View Problem&gt;
```

7. Testing Templates

Test with On-Demand Trigger

1. Create workflow with On-Demand trigger
2. Add JavaScript task to create mock event (`problem_link()` only evaluates under a Problem trigger, so link rendering cannot be tested this way):

```
export default async function() {
  return {
    // Field names match the dt.davis.problems record the Problem trigger
delivers
    mock_event: {
      "display_id": "P-TEST-001",
      "event.id": "-1234567890123456789_1790000000000V2",
      "event.name": "High response time on checkout-service",
      "event.category": "SLOWDOWN",
      "event.severity": "1",
      "event.status": "ACTIVE",
      "event.start": new Date().toISOString(),
      "affected_entity_ids": ["SERVICE-ABC123", "SERVICE-DEF456"],
      "root_cause_entity_id": "HOST-XYZ789"
    }
  };
}
```

3. Use mock data in notification:

```
message: |
  {{ result("mock_data").mock_event["event.category"] }} Alert
  {{ result("mock_data").mock_event["event.name"] }}
```

Preview Templates in Slack Block Kit Builder

Use https://app.slack.com/block-kit-builder to preview your block templates before deployment.

Query Notification Effectiveness

```
// Notification task performance
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-7d
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state.is_final == true
| summarize {executions = count(), avg_ms = avg(duration) / 1ms, p95_ms =
percentile(duration, 95) / 1ms}, by:{dt.automation_engine.action.function}
| sort p95_ms desc
| limit 20
```

```
// Action errors (connector / API failures). A malformed template fails before
the action runs - see the next query
// Data object corrected 08/12/2026. Workflow executions are NOT in `events`,
and there is no
// `automation.task.execution` / `automation.workflow.execution` event type in
any spelling — those
// filters matched nothing, silently, in 25 cells. AutomationEngine writes to
`dt.system.events` with
// `event.kind == "WORKFLOW_EVENT"` (5.7M records / 30d here), split by
`event.type`:
//   WORKFLOW_EXECUTION · TASK_EXECUTION · ACTION_EXECUTION ·
WORKFLOW_CREATED/UPDATED/DELETED
// Fields are the `dt.automation_engine.*` family:
//   .workflow.title / .workflow.id      (was workflow.name)
//   .task.name                          (was task.name)
//   .action.function / .action.app      (was task.type — e.g. run-javascript,
send-email,
//                                        snow-search-incidents)
//   .state                              (was task.status / execution.status)
//                                        values RUNNING · SUCCESS · ERROR ·
DISCARDED · SKIPPED
//                                        — note "FAILED" is NOT a value; it
is ERROR
//   .state.is_final                    true only on terminal records —
filter on it, otherwise a
//                                        single execution is counted once as
RUNNING and again as
//                                        SUCCESS/ERROR
//   .state_info                        (was task.error)  ·  duration (was
task.duration)
// Enumerate with:
//   fetch dt.system.events, from:-24h | filter event.kind == "WORKFLOW_EVENT"
| limit 1
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT"
| filter event.type == "ACTION_EXECUTION"
| filter dt.automation_engine.state == "ERROR"
| filter isNotNull(dt.automation_engine.state_info) and
dt.automation_engine.state_info != ""
| summarize errors = count(), by:{dt.automation_engine.action.function,
dt.automation_engine.state_info}
| sort errors desc
| limit 25
```

```
// Template (input-expression) errors
// A template that does not evaluate fails before any action runs, so it is
recorded on the
// WORKFLOW_EXECUTION record ("Error evaluating '&lt;input&gt;' in input.
..."), not on ACTION_EXECUTION.
fetch dt.system.events, from:-24h
| filter event.kind == "WORKFLOW_EVENT" and event.type == "WORKFLOW_EXECUTION"
| filter dt.automation_engine.state == "ERROR" and
contains(dt.automation_engine.state_info, "Error evaluating")
| summarize errors = count(), by:{dt.automation_engine.workflow.title,
dt.automation_engine.state_info}
| sort errors desc
| limit 25
```

Next Steps

With custom templates ready, implement auto-remediation:

Recommended Path

1. **WFLOW-07: Problem-Triggered Remediation** - Auto-remediation patterns
2. **WFLOW-08: JavaScript & HTTP Actions** - Custom integrations
3. **WFLOW-09: Security, Governance & Monitoring** - Best practices

Key Takeaways

- **Jinja2** enables dynamic, conditional content
- **Slack Block Kit** creates rich, interactive messages
- **Teams Adaptive Cards** provide structured formatting
- **Data enrichment** adds context from DQL queries
- **Test templates** with mock data before production, and watch for `Error evaluating` on workflow executions

---

Summary

In this notebook, you learned:

- Alert message design principles
- Jinja2 expressions, filters, and conditionals
- Slack Block Kit template structure
- Teams Adaptive Card formatting
- Data enrichment with DQL and entity lookups
- Template library patterns
- Testing strategies

References

- Workflow reference / Jinja expressions (DT docs)
- Notification actions umbrella (DT docs)
- Jinja2 template designer (Pallets)
- Slack Block Kit reference (Slack API)
- Slack Block Kit Builder (Slack)
- Teams Adaptive Cards (Microsoft Learn)

This notebook was AI-generated from community-submitted and publicly available sources. This notebook series is not officially supported by Dynatrace. Always verify information against official Dynatrace documentation.